

Figures: Worked Examples

The Vote Coin Protocol · White Paper v2.0, Revision 4 · every number in the 27 figures and on the cover, and how to recompute it · September 2026

Every figure in the white paper follows one sample batch of 8 ballots — the batch on the cover. They are not real votes. Every value in the figures is computed from them, or from the paper’s own stated numbers and the published parameters cited below, using the formats of Section 8.1, so anyone can check any figure with standard tools. Ballot **0515 0293**, traced in orange in the figures, is one of the 8. This companion replaces “Cover Figure: Worked Example” from Revision 1; the cover’s numbers have not changed.

Real salts never leave a Secure Node’s sealed audit log. These are printed only because the ballots are samples.

1. The sample batch

Item	Value
Node	Sample Node 01 (a sample)
Batch	0003 (sequence number 3)
Time window, published to the hour	09:00–10:00
Ballots	8 — far fewer than the 100 a real batch requires (Section 8.1); eight fit on a page
Root, anchored	e2fe3a46 a74352ad 0ebd28c3 95758307 45cb2b80 5b4acfae 00c394f1 f8d72a75

2. The rules used (white paper, Section 8.1)

Item	Rule
Ballot record R	UTF-8 text, one field per line, each line ending in a line feed: INDEX, a colon, a space and the eight index digits; then one line per contest in ballot order — its name, a colon, a space and the selection (NO SELECTION if left blank), spelled exactly as in the election definition.
Validation index	Eight digits drawn at random inside the Secure Node for each ballot, never repeated among that Node’s ballots in the election (a repeat is discarded and drawn again), and not derived from the voter, the session, the check-in or the time. Printed in OCR-B as two groups of four; R holds the eight digits without the space.
Salt s	32 random bytes (256 bits) for each ballot, from a NIST-approved random bit generator (SP 800-90A), kept only in the sealed audit log.
Fingerprint	SHA-256(R s) (FIPS 180-4). When the paper and the tablet’s record agree, this one value is both V_d and V_p, and it is the tracking code on the voter’s receipt.
Batch tree	IETF RFC 9162, Section 2.1.1, with SHA-256: leaf = SHA-256(0x00 fingerprint); node = SHA-256(0x01 left right). Fingerprints are sorted in ascending order first.
Batches	A batch closes after 60 minutes or 4,096 ballots, whichever comes first, but never with fewer than 100; each batch’s sorted fingerprints are published with its root.
Finality	An anchor is final once it is more than $3k/f = 129,600$ slots (36 hours) deep: $k = 2,160$, $f = 0.05$, slots of 1 second (Cardano mainnet genesis).
How values are printed	In full as 64 hexadecimal digits in eight groups of eight; shortened as the first and last eight digits (880d738d...08e537d6); the cover shows the first four.

3. The eight ballots

In tree order — sorted by fingerprint — with each ballot's place in the archive box (sorted by index) alongside.

#	Box	Index · selections · R	Salt s · fingerprint · leaf
1	3	3618 3202 CHOICE C · CHOICE B · YES R: 71 bytes	s 57306ce7 8f98f973 fe986687 17d552ae e64cf4b0 8c722a98 28b6f28e a621c740 fp 55eaae1d b87959fa c20311ad fb5e9290 90aa350f aae7cb99 b6f7831a 5a24cdea leaf a49c5e5f bcacce52 d714de56 52bc239d 4ddcd3bf 967f76f6 ec60da62 be634499
2	2	3090 4408 CHOICE B · CHOICE A · YES R: 71 bytes	s 1c5c036c a3fce2f6 87ce0004 c1a81ac6 bfba2770 cb45fc40 01d0e12b b8dc93a2 fp 5ad00c95 8d482c2c a26db7fb a6ee5229 05c43491 53c0fdfa 5904f11b 863b679a leaf ba4e01ae c72ac057 3dc6dc29 d4047f2a 98393acd cad673a6 8a5f13e1 345a383c
3	7	7702 8142 CHOICE A · CHOICE B · YES R: 71 bytes	s 25744722 67268764 f9c40cff 3c6f93a4 e4a2a2f8 f6fe6720 37312fac ae7f7e71 fp 756f385f 9db9c96c c55af062 132aa543 41dbb7d2 642f3e0e ae6a7c13 6fdbba4b8 leaf 9dbdec5e 3e9aef29 5cd254ea 0b339412 a1b8f165 8344e2d5 dbbeeb48 6e14c705
4	1	0515 0293 CHOICE A · CHOICE A · NO R: 70 bytes	s 03e747ee bf0adf01 32f9ad64 1548aca1 aad99672 599feebb b94196e0 cca4d40e fp 880d738d 9f978125 7ab513b3 12e5581f 71419543 b9c74f7e 5d6b7634 08e537d6 leaf 70c92ea4 d2b38741 d266303c c98fd4fa a616aaf1 7320afc5 54b38334 ce8bd14c
5	6	5307 1166 CHOICE A · CHOICE B · YES R: 71 bytes	s 41cd26de 7f1025bb d05c0f10 ec26dcd f c57c4681 a52e6dd2 86686ae0 7bc6e31b fp a1a467b8 8204e2b9 860c5e49 39851a17 32e2106f cc1c913c d1dfe336 2357600b leaf e499e5b3 dfda06b2 f9e3a44b 7d5bc954 7bc1acbb b1717748 cbe6bb22 2e4b354a
6	8	9984 2665 CHOICE A · CHOICE B · YES R: 71 bytes	s 7736d58c 8d73883c 0b422b5b 8d5a53fd b3ac44f3 642841d6 4137d136 8d897171 fp b245d512 657427cb d57920b9 20dcd7bf 26fee31f d56f0543 cb8b07df 1929f4df leaf c80baf81 d900a904 356586ab 8eafaf37 4fb5c79a a8ea9996 3aefff38 e3c45217
7	4	4199 9390 CHOICE A · CHOICE B · NO R: 70 bytes	s ace827c2 d4122f84 64fcbdea 7de51672 37fe0947 202dc9f1 bf7f8abf 7a1e94e0 fp d44e41fa 67da1fb0 5f33b7d6 644af88c 017fe74f e78c2e47 0bcacdc5 c73aec54 leaf 0dfd9eea 77dd1f62 0618d01f 0066dac6 6f77892b 17f9fd1f a429c72f 559d3731
8	5	4308 5720 CHOICE C · CHOICE A · NO SELECTION R: 80 bytes	s 86e6d63f 30da6b38 2e586d7b 9e713384 2c19f7ad a893d0ef 284927ef c220e530 fp f8bbfe7b db716361 d64920b2 a67cf78b acd1449e eb4a66ce fce0a293 cf3dbb10 leaf f1d60f5f 99bc308d a7a8274e 380c470d b400005a ab8608a3 b3016457 11a818cf

4. The records, byte for byte

Each record R as UTF-8 text; ␣ marks the line feed (byte 0x0A) that ends every line. Tree order.

```

1 INDEX: 36183202␣CONTEST 1: CHOICE C␣CONTEST 2: CHOICE B␣MEASURE 3: YES␣ (71 bytes)
2 INDEX: 30904408␣CONTEST 1: CHOICE B␣CONTEST 2: CHOICE A␣MEASURE 3: YES␣ (71 bytes)
3 INDEX: 77028142␣CONTEST 1: CHOICE A␣CONTEST 2: CHOICE B␣MEASURE 3: YES␣ (71 bytes)
4 INDEX: 05150293␣CONTEST 1: CHOICE A␣CONTEST 2: CHOICE A␣MEASURE 3: NO␣ (70 bytes)
5 INDEX: 53071166␣CONTEST 1: CHOICE A␣CONTEST 2: CHOICE B␣MEASURE 3: YES␣ (71 bytes)
6 INDEX: 99842665␣CONTEST 1: CHOICE A␣CONTEST 2: CHOICE B␣MEASURE 3: YES␣ (71 bytes)
7 INDEX: 41999390␣CONTEST 1: CHOICE A␣CONTEST 2: CHOICE B␣MEASURE 3: NO␣ (70 bytes)
8 INDEX: 43085720␣CONTEST 1: CHOICE C␣CONTEST 2: CHOICE A␣MEASURE 3: NO SELECTION␣ (80 bytes)

```

Ballot 0515 0293's record in hexadecimal (70 bytes):

```

49 4e 44 45 58 3a 20 30 35 31 35 30 32 39 33 0a 43 4f 4e 54 45 53 54 20 31 3a 20 43 48 4f 49 43
45 20 41 0a 43 4f 4e 54 45 53 54 20 32 3a 20 43 48 4f 49 43 45 20 41 0a 4d 45 41 53 55 52 45 20
33 3a 20 4e 4f 0a

```

5. Two orders, neither of them time

Order	The eight ballots
In the archive box, by index (Figure 7)	0515 0293, 3090 4408, 3618 3202, 4199 9390, 4308 5720, 5307 1166, 7702 8142, 9984 2665
In the tree, by fingerprint (Figure 12)	55eaae1d < 5ad00c95 < 756f385f < 880d738d < a1a467b8 < b245d512 < d44e41fa < f8bbfe7b
Ballot 0515 0293	1 of 8 by index; 4 of 8 by fingerprint

6. The tree

Leaves are in the ballot table above. Each node is SHA-256(0x01 || left || right).

Level	Values
Level 1 (four nodes)	43e553d9 e8f64a93 95db961a cd71f82c 4b96aacf 9040c59e 4e40089b 86826a9f c8b8b6d0 9e049bcd ad61b091 1b469538 ba03c4d0 65ba869e 8fabbb19 e03f1d54 d13516a0 04fbff74 039c5c6f e38be814 68b094f8 de192b0f cce2e6fa 0ff1f470 881ac392 8498b451 ac1fe948 f46edd32 e2a3485a d1a887a1 e8a12571 c75a925c
Level 2 (two nodes)	b83a4efc 8cb724c6 a7b15f61 8b10ae60 3abefa9a 9bf0447c 563ecd44 bc639c22 00b17d9f 446fb73b f316fe02 4150f373 f11b8457 8060b069 9b43ef40 b24667b8
Root (anchored)	e2fe3a46 a74352ad 0ebd28c3 95758307 45cb2b80 5b4acfae 00c394f1 f8d72a75

7. Ballot 0515 0293, step by step (Figures 1, 2, 11, 25)

Step	Value
Record R	70 bytes: INDEX: 05150293␣ CONTEST 1: CHOICE A␣ CONTEST 2: CHOICE A␣ MEASURE 3: NO␣
Salt s	03e747ee bf0adf01 32f9ad64 1548aca1 aad99672 599feebb b94196e0 cca4d40e
Hash input R s	70 + 32 = 102 bytes
V_d = SHA-256(R s), from the tablet	880d738d 9f978125 7ab513b3 12e5581f 71419543 b9c74f7e 5d6b7634 08e537d6
V_p = SHA-256(R s), R rebuilt by OCR from the printed ballot	880d738d 9f978125 7ab513b3 12e5581f 71419543 b9c74f7e 5d6b7634 08e537d6
S	1: V_d = V_p, the paper and the record agree
Tracking code on the receipt	the fingerprint itself — never the index
Leaf = SHA-256(0x00 fingerprint)	70c92ea4 d2b38741 d266303c c98fd4fa a616aaf1 7320afc5 54b38334 ce8bd14c

What if (Figure 11): a tablet that printed CHOICE A for Contest 2 but recorded CHOICE B would give

Record the tablet kept	V_d'	S
INDEX: 05150293␣ CONTEST 1: CHOICE A␣ CONTEST 2: CHOICE B␣ MEASURE 3: NO␣	619d2d8c c9fa14b2 489dbe01 9a50f3e9 2b416d2c ba041ae3 69a17e44 2de1257d	0 — caught

8. Proving ballot 0515 0293 was anchored (Figure 16)

Three sibling hashes — 96 bytes — take the leaf to the root. Each step is SHA-256(0x01 || left || right).

Step	Sibling	Left right → result
1	on the left	left 9dbdec5e 3e9aef29 5cd254ea 0b339412 a1b8f165 8344e2d5 dbbeeb48 6e14c705 right 70c92ea4 d2b38741 d266303c c98fd4fa a616aaf1 7320afc5 54b38334 ce8bd14c → c8b8b6d0 9e049bcd ad61b091 1b469538 ba03c4d0 65ba869e 8fabbb19 e03f1d54
2	on the left	left 43e553d9 e8f64a93 95db961a cd71f82c 4b96aacf 9040c59e 4e40089b 86826a9f right c8b8b6d0 9e049bcd ad61b091 1b469538 ba03c4d0 65ba869e 8fabbb19 e03f1d54 → b83a4efc 8cb724c6 a7b15f61 8b10ae60 3abefa9a 9bf0447c 563ecd44 bc639c22
3	on the right	left b83a4efc 8cb724c6 a7b15f61 8b10ae60 3abefa9a 9bf0447c 563ecd44 bc639c22 right 00b17d9f 446fb73b f316fe02 4150f373 f11b8457 8060b069 9b43ef40 b24667b8 → e2fe3a46 a74352ad 0ebd28c3 95758307 45cb2b80 5b4acfae 00c394f1 f8d72a75

The last result equals the anchored root, e2fe3a46...f8d72a75.

9. Proof sizes (Figures 16, 17)

Ballots in the batch, n	At most $\lceil \log_2 n \rceil$ hashes	Bytes (32 each)
8	3	96
100	7	224
500	9	288
1,000	10	320
1,500	11	352
4,096	12	384
1,048,576	20	640

Checked by building every inclusion path exactly as RFC 9162 §2.1.3.1 defines it, for every n up to 4,096; the longest path always equals $\lceil \log_2 n \rceil$.

10. Batch rules and finality (Figures 2, 12, 17)

$3k/f = 3 \times 2,160 \div 0.05 = 129,600$ slots
 $129,600 \text{ slots} \times 1 \text{ s} = 129,600 \text{ s} = 36 \text{ hours}$
 batch closes: 60 minutes or 4,096 ballots, never fewer than 100

11. Capacity per Node (Figures 5, 10, 18)

Printers failed	Working	Printed per hour	At or above the Phase I peak of 500?
0	5	$5 \times 300 = 1,500$	yes
1	4	$4 \times 300 = 1,200$	yes
2	3	$3 \times 300 = 900$	yes
3	2	$2 \times 300 = 600$	yes
4	1	$1 \times 300 = 300$	no

$T = \min(\text{printing}, \text{filing}) = \min(1,500, 1,000) = 1,000$ an hour
 $\text{filing} \div \text{peak} = 1,000 \div 500 = 2\times$; $\text{printing} \div \text{peak} = 1,500 \div 500 = 3\times$
 stress test: $1,000$ an hour $\times 8$ hours = $8,000$ ballots

Rates are the paper's own design values (Sections 4.3, 12.2–12.3): about 300 ballots an hour per printer, 1,000 an hour filing, a Phase I peak of 200–500 an hour.

12. The Phase I gate: the rule of three (Figure 23)

$n \geq \ln 0.05 \div \ln(1 - 0.00001) = 299571.7$, rounded up to 299,572 $\rightarrow$ the gate asks for 300,000
 8,000 ballots alone: $1 - 0.05^{(1/8,000)} = 0.0003744 = 0.0374\%$
 300,000 ballots: $1 - 0.05^{(1/300,000)} = 9.9857e-6$, below 0.001%
 300,000 at 1,000 an hour = 300 hours of robotic testing

Hanley & Lippman-Hand (1983). The bound treats each reconciliation as an independent trial with one error rate; it bounds random errors, not a targeted attack.

13. Validation-index repeats (Figure 7)

Draws n	Expected repeats $n(n-1)/2N$	$P \approx 1 - e^{-(n(n-1)/2N)}$	Exact product
1,000	0.0050	0.50%	0.4983%
5,000	0.1250	11.75%	11.7483%
10,000	0.5000	39.34%	39.3449%

$N = 100,000,000$ possible indices. A repeat is never used: the draw is discarded and made again. Formula: Menezes, van Oorschot & Vanstone, Handbook of Applied Cryptography, Fact 2.27.

14. Objection 7: the arithmetic of being noticed (Figure 26)

Voters given a flipped ballot, n	$P = 1 - (1 - 0.066)^n$
1	6.6%
5	28.9%
11	52.8%
20	74.5%
50	96.7%
100	99.9%

$d = 0.066$: 6.6% of participants told a poll worker, without prompts (Bernhard et al., 2020). Assumes voters act independently and one report is acted on. A 20-vote margin is overturned by 11 flips, since each flip moves it by 2.

15. Throughput and salts (Figures 11, 14, 17)

3.5 transactions a second $\times 3,600 = 12,600$ an hour (observed on mainnet; Input Output Global, 2026)
 a 32-byte salt: $2^{256} =$
 $115792089237316195423570985008687907853269984665640564039457584007913129639936 \approx 1.16 \times 10^{77}$
 possible values

16. The one hypothetical example (Figure 21)

Figure 21 shows one Node's day with invented numbers, labelled as an example: 962 Vote Coins burned + 23 provisional + 15 regular ballots without the card = 1,000 voters' ballots cast; 12 spoiled ballots are logged apart. They show the arithmetic only; they are not data.

17. Figure by figure

Figure	Title	Page in the FINAL	Values from sections
1	How to read the figures	5	3, 4, 7
2	The life of one ballot	8	7, 6, 10
3	The five pillars of verification	12	7, 6
4	Public money in, nothing of value out	15	none computed (text and law only)
5	Inside a Secure Node	17	11
6	The air gap, and where data crosses	18	6, 11
7	Filed by batch and index	20	5, 13
8	Eight threats, eight answers	25	none computed (text and law only)
9	Three keys, three institutions	28	none computed (text and law only)
10	Degraded, never suspended	30	11
11	The reconciliation check, ballot 0515 0293	33	7
12	How a batch becomes one root	34	3, 6, 10
13	Three outcomes, one rule	36	7, 12
14	Public, sealed, never recorded	40	3, 6, 15
15	From consultation to certification	43	none computed (text and law only)
16	Proving ballot 0515 0293 was anchored	48	8, 9
17	Local first, anchored later	50	10, 11, 15
18	Capacity per Node	52	11

Figure	Title	Page in the FINAL	Values from sections
19	Enroll once, vote with a card	54	none computed (text and law only)
20	Audit any batch	56	3, 6, 7
21	A Vote Coin's life	58	16
22	Five invariants	64	7
23	Four phases, four gates	66	12, 11
24	Where the certification line falls	69	none computed (text and law only)
25	One representation	73	7
26	If a tablet lies	75	14
27	The sample batch, in full	87	3, 4, 6

18. Check it yourself

The reader kit that comes with this document — `figdata.json` and `verify_figures.py` — recomputes every value above with the Python 3 standard library alone and prints PASS or FAIL for each: `python3 verify_figures.py`. The lines below are the core of it, short enough to type.

```
import hashlib
H = lambda b: hashlib.sha256(b).digest()
R = b'INDEX: 05150293\nCONTEST 1: CH0ICE A\nCONTEST 2: CH0ICE A\nMEASURE 3: N0\n'
s = bytes.fromhex('03e747eebf0adf0132f9ad641548aca1aad99672599feebbb94196e0cca4d40e')
print(H(R + s).hex()) # 880d738d...08e537d6

# fps = the eight fingerprints, in tree order, as bytes.fromhex(...) values
leaf = [H(b'\x00' + f) for f in fps]
l1 = [H(b'\x01' + leaf[i] + leaf[i + 1]) for i in (0, 2, 4, 6)]
l2 = [H(b'\x01' + l1[0] + l1[1]), H(b'\x01' + l1[2] + l1[3])]
print(H(b'\x01' + l2[0] + l2[1]).hex()) # e2fe3a46...f8d72a75
```

How these values were checked. Computed in Python, then recomputed independently with Node.js, with OpenSSL and GNU sha256sum on the exact binary inputs, and the arithmetic with bc; all agree. The tree code reproduces the published RFC 9162 reference roots for trees of one to eight leaves, and SHA-256 reproduces NIST's published example value. Three independent reviewers recomputed them again in September 2026.

19. Sources

NIST FIPS 180-4, Secure Hash Standard (2015) — <https://doi.org/10.6028/NIST.FIPS.180-4>

IETF RFC 9162, Certificate Transparency Version 2.0, §2.1 (2021) — <https://datatracker.ietf.org/doc/html/rfc9162>

NIST SP 800-90A Rev. 1 (2015) — <https://doi.org/10.6028/NIST.SP.800-90Ar1>

ISO 1073-2:1976, Character set OCR-B — <https://www.iso.org/standard/5568.html>

Reference test values, transparency-dev/merkle — <https://github.com/transparency-dev/merkle/blob/main/testonly/constants.go>

Cardano mainnet genesis parameters — <https://book.world.dev.cardano.org/environments/mainnet/shelley-genesis.json>

Cardano documentation, chain confirmation versus transaction confirmation — <https://docs.cardano.org/about-cardano/learn/chain-confirmation-versus-transaction-confirmation>

Input Output Global, What is Leios (throughput figures) — <https://leios.cardano-scaling.org/docs/what-is-leios/>

Hanley & Lippman-Hand, JAMA 249(13), 1983 (author copy) — https://jhanley.biostat.mcgill.ca/c607/ch08/zero_numerator.pdf
Bernhard et al., IEEE S&P 2020 (author copy) — <https://jhalderm.com/pub/papers/bmd-verifiability-sp20.pdf>
Menezes, van Oorschot & Vanstone, Handbook of Applied Cryptography, ch. 2 — <https://cacr.uwaterloo.ca/hac/about/chap2.pdf>